LU Factorization

Geometric Algorithms
Lecture 12

CAS CS 132

Practice Problem

$$\begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}$$

Determine the inverse of the above matrix by both using the explicit formula and by reducing a matrix. Compare the results

Answer

$$\begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}$$

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$= \frac{1}{7-6} \begin{bmatrix} 7 & -2 \\ -3 & 1 \end{bmatrix} = \begin{bmatrix} 7 & -2 \\ -3 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 1 & 0 \\ 3 & 7 & 0 & 1 \end{bmatrix} \overset{-3 \quad -6 \quad -3}{\sim} \overset{-2 \quad +6 \quad -2}{\begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & -3 & 1 \end{bmatrix}} \sim \begin{bmatrix} 1 & 0 & 7 & -2 \\ 0 & 1 & -3 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix} \begin{bmatrix} 7 & -2 \\ -3 & 1 \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix} \begin{bmatrix} 7 \\ -3 \end{bmatrix} & \begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix} \begin{bmatrix} -2 \\ 1 \end{bmatrix} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Objectives

- Finish up our discussion of the **algebra of matrix inverses**
- Motivate **matrix factorization** in general, and the LU factorization in specific
- Recall elementary row operations and connect them to matrices
- Look at the **LU factorization**, how to find it, and how to use it

Algebra of Matrix Inverses

Recall: Matrix Inverses

Recall: Matrix Inverses

Df. An **inverse** of $A \in \mathbb{R}^{n \times n}$ is a matrix $B \in \mathbb{R}^{n \times n}$ such that

$$AB = I_n \text{ and } BA = I_n$$

Recall: Matrix Inverses

Df. An **inverse** of $A \in \mathbb{R}^{n \times n}$ is a matrix $B \in \mathbb{R}^{n \times n}$ such that

$$AB = I_n \text{ and } BA = I_n$$

A is **invertible** if it has an inverse. Otherwise it is **singular**

Recall: Challenge Problem

Recall: Challenge Problem

Df. B is **right inverse** of A if $AB = I$

Recall: Challenge Problem

Df. B is **right inverse** of A if $AB = I$

Df. B is a **left inverse** of A if $BA = I$

Recall: Challenge Problem

Df. B is **right inverse** of A if $AB = I$

Df. B is a **left inverse** of A if $BA = I$

Thm. If A has a right inverse B, then B is also a left inverse A

Recall: Challenge Problem

Df. B is **right inverse** of A if $AB = I$

Df. B is a **left inverse** of A if $BA = I$

Thm. If A has a right inverse B, then B is also a left inverse A

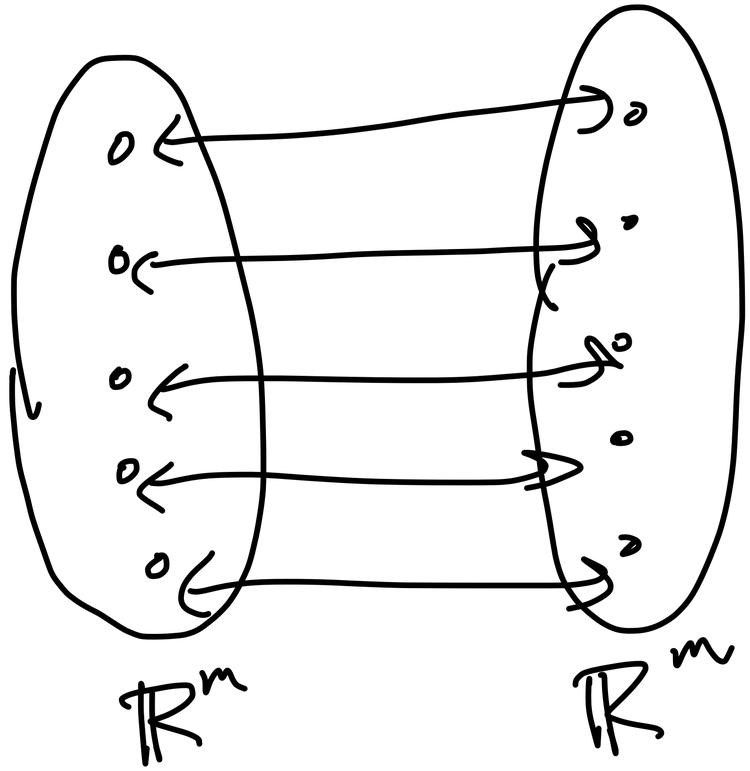
We only need to show A has a right/left inverse to show it is invertible

Inverses are invertible

Thm. For an invertible matrix A, the matrix A^{-1} is invertible

$$A^{-1}B = I$$

$$A^{-1}A = I$$



Transposes are invertible

Thm. For an invertible matrix A, the matrix A^T is invertible

$$A^T B = I$$

$$(B^T A)^T = I$$

$$B^T A = I$$

$$(AB)^T = B^T A^T$$

$$B^T A A^{-1} = I A^{-1} \qquad (B^T)^T = B$$

$$B^T = A^{-1}{}^T$$

$$B = (A^{-1})^T$$

Transposes are invertible

Thm. For an invertible matrix A, the matrix A^T is invertible

$$A^T \left(A^{-1} \right)^T = \left(A^{-1} A \right)^T$$

$$= I^T$$

$$= I$$

$$(AB)^T = B^T A^T$$

$$\left(A^T\right)^{-1} = \left(A^{-1}\right)^T$$

Products of invertibles are invertible

Thm. For a invertible matrices $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times n}$, the matrix AB is invertible

$(AB)C = I$

$C = A^{-1}B^{-1}$

$ABA^{-1}B^{-1}$

$AB \neq BA$

$(AB)^{-1} = B^{-1}A^{-1}$

$(AB)(B^{-1}A^{-1}) =$

$ABB^{-1}A^{-1} =$

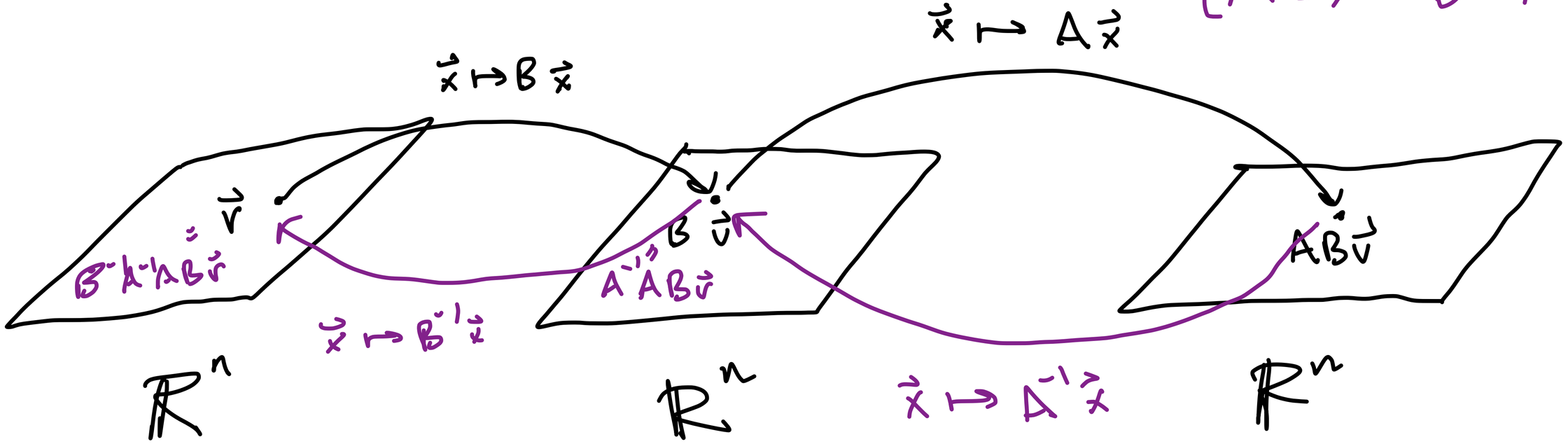
$AIA^{-1} =$

$AA^{-1} = I$

Products of invertibles are invertible

Thm. For a invertible matrices $A \in \mathbb{R}^{n\times n}$ and $B \in \mathbb{R}^{n\times n}$, the matrix AB is invertible

$(AB)^{-1} = B^{-1}A^{-1}$



Question

Suppose that A *is a* $n \times n$ *invertible matrix such that* $A = A^T$ *and* B *is a* $m \times n$ *matrix*

symmetric

Simplify the expression $A(BA^{-1})^T$ *using the algebraic properties we've seen*

$$A(BA^{-1})^T = A(A^{-1})^T B^T = A(A^T)^{-1} B^T$$

$$= A A^{-1} B^T$$

$$= I B^T = \boxed{B^T}$$

Answer: B^T

$$A(BA^{-1})^T$$

$$A = A^T$$

Invertible Matrix Theorem

The Point

Q. How do we know if a square matrix is invertible?

A. *Every* perspective we've taken so far can help us answer this question

The Invertible Matrix Theorem

The following are logically equivalent:

1. A is invertible
2. A^T is invertible
3. $A\mathbf{x} = \mathbf{b}$ has at least one solution for any $\mathbf{b}$
4. $A\mathbf{x} = \mathbf{b}$ has at most one solution for any $\mathbf{b}$
5. $A\mathbf{x} = \mathbf{b}$ has a unique solution for any $\mathbf{b}$
6. A has n pivots (per row and per column)
7. A is row equivalent to I
8. $A\mathbf{x} = \mathbf{0}$ has only the trivial solution
9. The columns of A are linearly independent
10. The columns of A span $\mathbb{R}^n$
11. The linear transformation $\mathbf{x} \mapsto A\mathbf{x}$ is onto
12. $\mathbf{x} \mapsto A\mathbf{x}$ is one-to-one
13. $\mathbf{x} \mapsto A\mathbf{x}$ is a one-to-one correspondence
14. $\mathbf{x} \mapsto A\mathbf{x}$ is invertible

Invertible Matrix Theorem

Thm. For any invertible matrix $A \in \mathbb{R}^{n \times n}$

1. A^T is invertible

Invertible Matrix Theorem

Thm. For any invertible matrix $A \in \mathbb{R}^{n \times n}$

2. $A\mathbf{x} = \mathbf{b}$ has at <u>least</u> one solution for every $\mathbf{b}$
3. $A\mathbf{x} = \mathbf{b}$ has at <u>most</u> one solution for every $\mathbf{b}$
4. $A\mathbf{x} = \mathbf{b}$ has at <u>exactly</u> one solution for every $\mathbf{b}$

Invertible Matrix Theorem

Thm. For any invertible matrix $A \in \mathbb{R}^{n \times n}$

5. A has a pivot in every column
6. A has a pivot in every row
7. A is row equivalent to I_n

Invertible Matrix Theorem

Thm. For any invertible matrix $A \in \mathbb{R}^{n \times n}$

8. The columns of A are linearly independent
9. The columns of A span $\mathbb{R}^n$
10. $A\mathbf{x} = \mathbf{0}$ has only the trivial solution

Invertible Matrix Theorem

Thm. For any invertible matrix $A \in \mathbb{R}^{n \times n}$

11. The linear transformation $\mathbf{x} \mapsto A\mathbf{x}$ is onto
12. $\mathbf{x} \mapsto A\mathbf{x}$ is one-to-one
13. $\mathbf{x} \mapsto A\mathbf{x}$ is a one-to-one correspondence
14. $\mathbf{x} \mapsto A\mathbf{x}$ is invertible

Taking Stock: IMT

The following are logically equivalent:

1. A is invertible
2. A^T is invertible
3. $A\mathbf{x} = \mathbf{b}$ has at least one solution for any $\mathbf{b}$
4. $A\mathbf{x} = \mathbf{b}$ has at most one solution for any $\mathbf{b}$
5. $A\mathbf{x} = \mathbf{b}$ has a unique solution for any $\mathbf{b}$
6. A has n pivots (per row and per column)
7. A is row equivalent to I
8. $A\mathbf{x} = \mathbf{0}$ has only the trivial solution
9. The columns of A are linearly independent
10. The columns of A span $\mathbb{R}^n$
11. The linear transformation $\mathbf{x} \mapsto A\mathbf{x}$ is onto
12. $\mathbf{x} \mapsto A\mathbf{x}$ is one-to-one
13. $\mathbf{x} \mapsto A\mathbf{x}$ is a one-to-one correspondence
14. $\mathbf{x} \mapsto A\mathbf{x}$ is invertible

These all express the **same thing**

!! only for square matrices !!

We get a lot of information for free

We get a lot of information for free

Thm. If A is square, then

A **is 1–1** if and only if A **is onto**

We get a lot of information for free

Thm. If A is square, then

A **is 1–1** if and only if A **is onto**

We only need to check one of these

We get a lot of information for free

Thm. If A is square, then

A **is 1–1** if and only if A **is onto**

We only need to check one of these

Warning. Remember this only applies square matrices

We get a lot of information for free

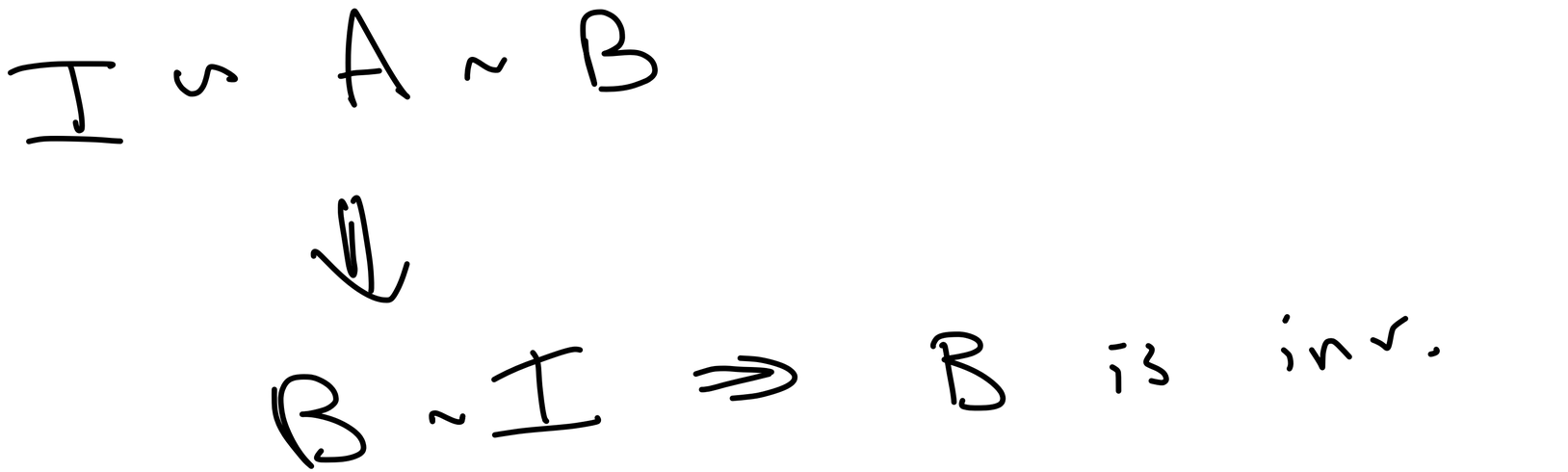
Theorem. If A is square, then

A **is invertible** $\equiv$ $A\mathbf{x} = \mathbf{0}$ **implies** $\mathbf{x} = \mathbf{0}$

Invertibility is completely determined by what A *sends to* $\mathbf{0}$

Question

If A is invertible, and B is row equivalent to A, is B is also invertible?



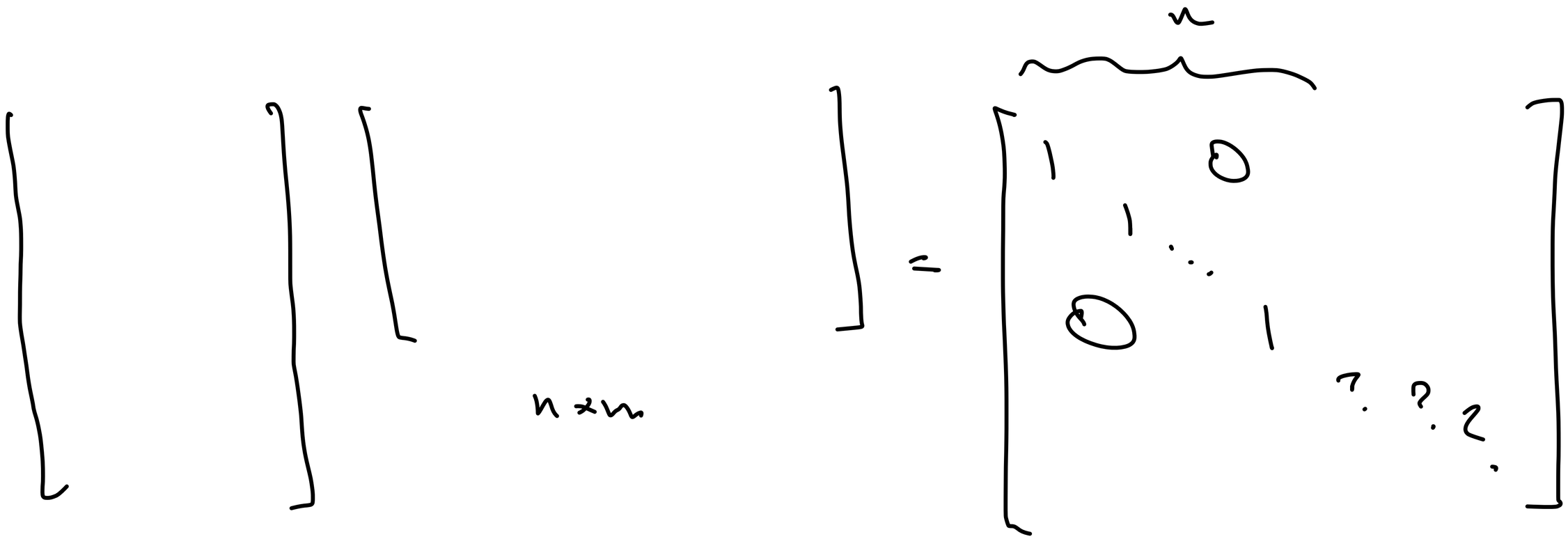
Challenge Problem

If $[\mathbf{a}_1 \;\; \mathbf{a}_2 \;\; \mathbf{a}_3]$ *is invertible, is*
$\left[(\mathbf{a}_1 + \mathbf{a}_2 - 2\mathbf{a}_3) \;\; (\mathbf{a}_2 + 5\mathbf{a}_3) \;\; \mathbf{a}_3\right]$ *also invertible?*

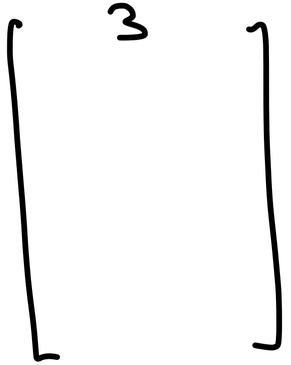
$$\begin{bmatrix} & \\ & \end{bmatrix}_{m \times n} \begin{bmatrix} & \\ & \\ & \end{bmatrix}_{n \times m} = I_m$$

$m < n$

$$\begin{bmatrix} & & \\ & & \end{bmatrix}_{2 \times 3} \begin{bmatrix} & \\ & \\ & \end{bmatrix}_{3 \times 2} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

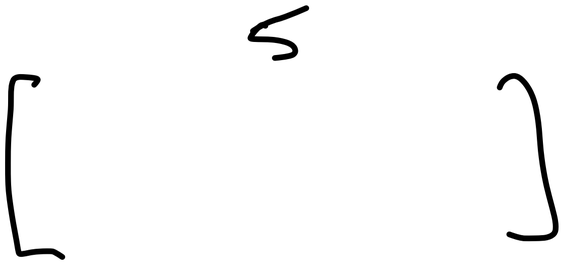

n
1
0
1
0
1
? ? ?
n × n
m × n
m × m
m > n

$$A \in \mathbb{R}^{5\times 3}$$



$$\vec{x} \mapsto A\vec{x} : \mathbb{R}^3 \to \mathbb{R}^5$$

$$B \in \mathbb{R}^{3\times 5}$$



$$\vec{x} \mapsto B\vec{x} : \mathbb{R}^5 \to \mathbb{R}^3$$

$$\vec{x} \mapsto AB\vec{x} : \mathbb{R}^5 \to \mathbb{R}^5$$

$$\vec{x} \mapsto BA\vec{x} : \mathbb{R}^3 \to \mathbb{R}^3$$

LU Factorization

Matrix Factorization

Matrix Factorization

A **factorization** of a matrix A is an equation which expresses A as a product of one or more matrices, e.g.,

$$A = BC$$

Reasons to Factor

Reasons to Factor

Writing A as the product of multiple matrices can

Reasons to Factor

Writing A as the product of multiple matrices can

» make computing with A faster

Reasons to Factor

Writing A as the product of multiple matrices can

- make computing with A faster
- make working with A easier

Reasons to Factor

Writing A as the product of multiple matrices can

- make computing with A faster
- make working with A easier
- expose important information about A

Reasons to Factor

Writing A as the product of multiple matrices can

- make computing with A faster LU Decomposition
- make working with A easier
- expose important information about A

The Problem

The Problem

Q. Solve the equations

The Problem

Q. Solve the equations

$$A\mathbf{x} = \mathbf{b}_1 \quad , \quad A\mathbf{x} = \mathbf{b}_2 \quad \ldots \quad A\mathbf{x} = \mathbf{b}_{k-1} \quad , \quad A\mathbf{x} = \mathbf{b}_k$$

The Problem

Q. Solve the equations

$$A\mathbf{x} = \mathbf{b}_1 \quad , \quad A\mathbf{x} = \mathbf{b}_2 \quad \ldots \quad A\mathbf{x} = \mathbf{b}_{k-1} \quad , \quad A\mathbf{x} = \mathbf{b}_k$$

We want to solve *several* matrix equations over the same matrix

The Problem

$$A\begin{bmatrix} \vec{x}_1 & \cdots & \vec{x}_n \end{bmatrix} = \begin{bmatrix} A\vec{x}_1 & \cdots & A\vec{x}_n \end{bmatrix}$$

$$A\vec{x}_1 = \vec{b}_1$$
$$A\vec{x}_2 = \vec{b}_2$$
$$\vdots$$
$$A\vec{x}_n = \vec{b}_n$$

Q. Solve for X in the equation

$$AX = B = \begin{bmatrix} \vec{b}_1 & \cdots & \vec{b}_n \end{bmatrix}$$

where X and B are matrices of appropriate dimension

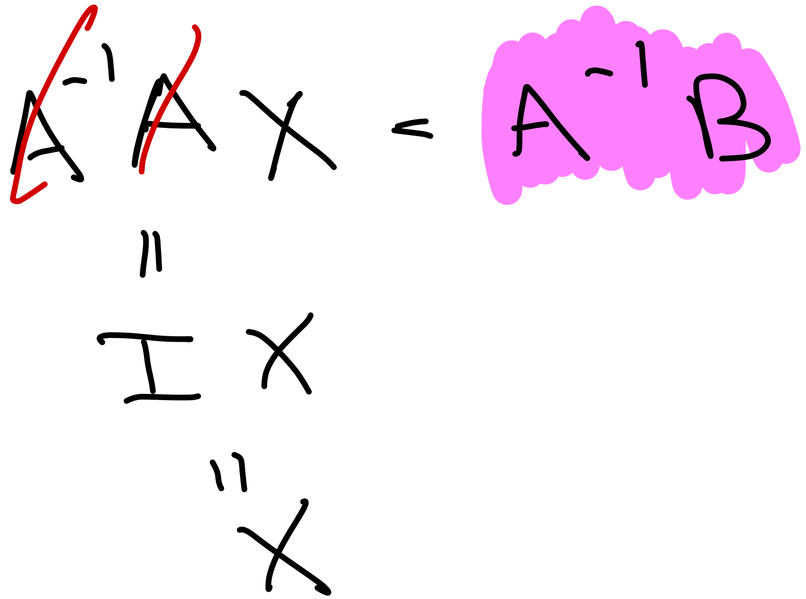
$$\begin{bmatrix} \vec{x}_1 & , \cdots & , x_n \end{bmatrix}$$

This is the same question

The Problem

Q. Solve $AX = B$

A. If A is *invertible*, then we have a solution:



The Problem

Q. Solve $AX = B$

A. If A is *invertible*, then we have a solution:

What if A^{-1} is not invertible?
Even if it is, can we do it faster?

LU Factorization at a High Level

Given $A \in \mathbb{R}^{m \times n}$, we'll write A as

echelon form of A

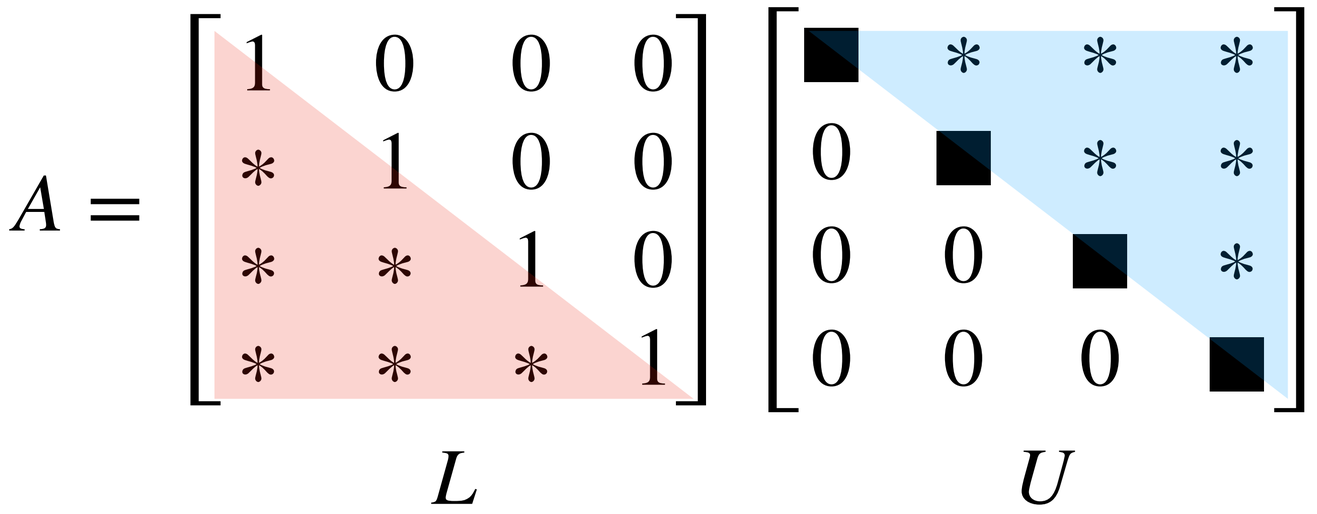
$$A = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & * & 1 & 0 \\ * & * & * & 1 \end{bmatrix}}_{L} \underbrace{\begin{bmatrix} \blacksquare & * & * & * & * \\ 0 & \blacksquare & * & * & * \\ 0 & 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_{U}$$

(This still works for non-square matrices)

What are "L" and "U"?

L stands for "lower" as in *lower triangular*

U stands for "upper" as in *upper triangular*



The Fundamental Question

$$A = LU$$ echelon form of A

The Fundamental Question

$$A = LU$$ echelon form of A

We know how to build U (forward phase of Gaussian elimination)

The Fundamental Question

$$A = LU$$ echelon form of A

We know how to build U (forward phase of Gaussian elimination)

How do we build L?

The Fundamental Question

$$A = LU$$ echelon form of A

We know how to build U (forward phase of Gaussian elimination)

How do we build L?

The idea. L "implements" the row operations of the forward phase

Elementary Matrices

Recall: Elementary Row Operations

scaling	multiply a row by a number
interchange	switch two rows
replacement	add a scaled equation to another

Recall: Elementary Row Operations

scaling	multiply a row by a number
interchange	switch two rows
replacement	add a scaled equation to another

Key observation: Elementary row operations are *linear transformations*

Scaling

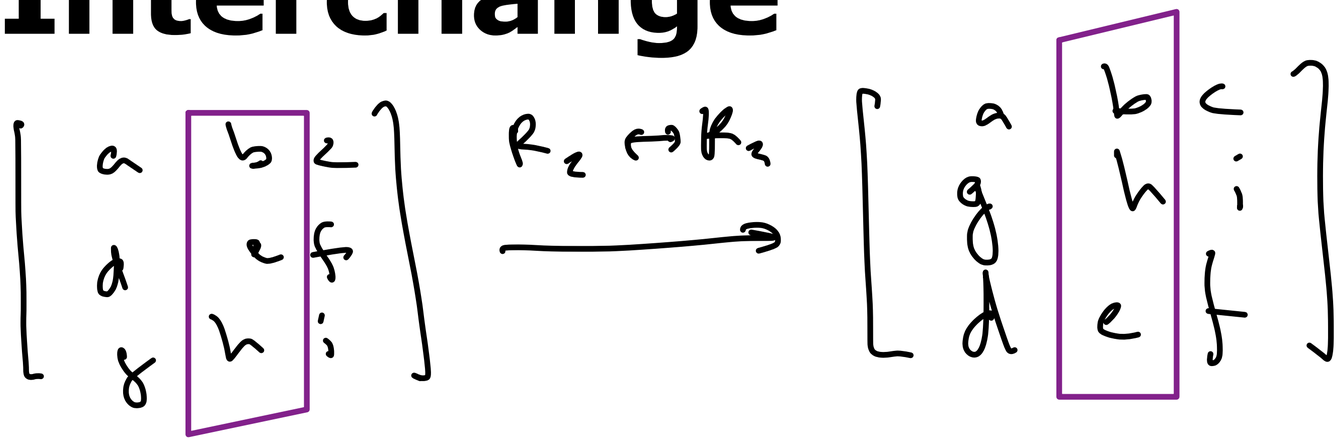
$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \xrightarrow{R_2 \leftarrow 5R_2} \begin{bmatrix} a & b & c \\ 5d & 5e & 5f \\ g & h & i \end{bmatrix}$$

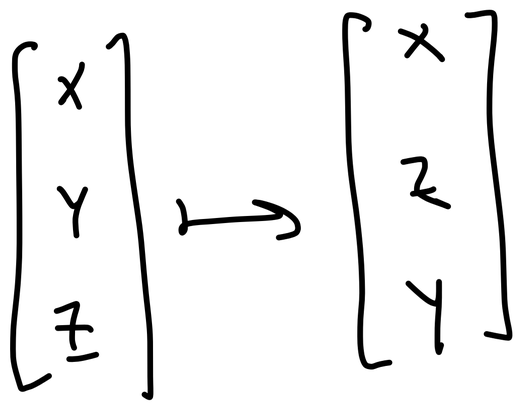
$$\begin{bmatrix} x \\ y \\ z \end{bmatrix} \mapsto \begin{bmatrix} x \\ 5y \\ z \end{bmatrix}$$

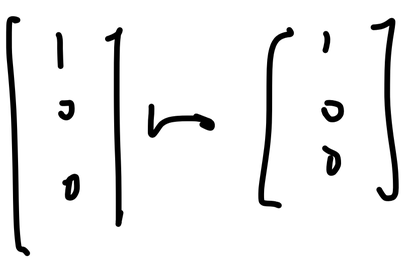
$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \qquad \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \mapsto \begin{bmatrix} 0 \\ 5 \\ 0 \end{bmatrix}$$

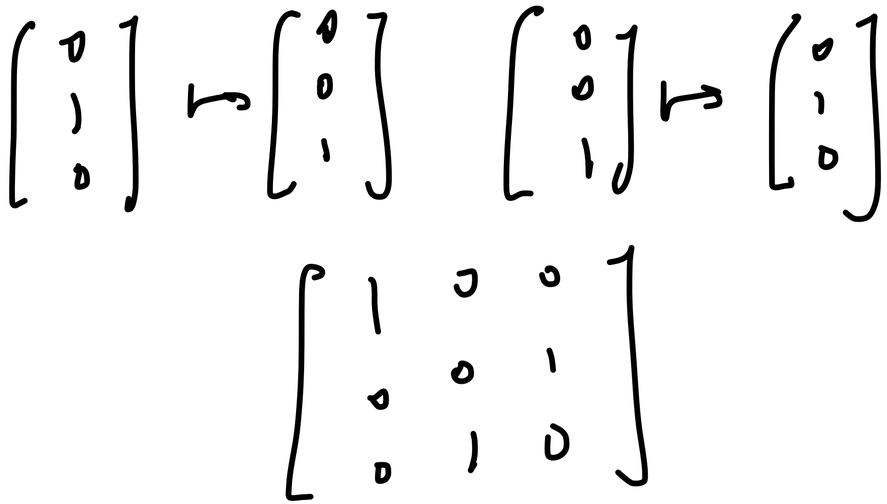
$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} = \begin{bmatrix} a & b & c \\ 5d & 5e & 5f \\ g & h & i \end{bmatrix} \qquad \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \mapsto \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \qquad \begin{bmatrix} 1 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Interchange









Replacement

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \xrightarrow{R_2 \leftarrow R_2 - 3R_1} \begin{bmatrix} a & b & c \\ d-3a & e-3b & f-3c \\ g & h & i \end{bmatrix}$$

$$\begin{bmatrix} a \\ d \\ g \end{bmatrix} \mapsto \begin{bmatrix} a \\ d-3a \\ g \end{bmatrix}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

General Elementary Scaling Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & k & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

General Elementary Scaling Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & k & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_i \leftarrow kR_i$ is equivalent to multiplying by the identity matrix with $A_{ii} = k$

General Replacement Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ k & 0 & 0 & 1 \end{bmatrix}$$

General Replacement Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ k & 0 & 0 & 1 \end{bmatrix}$$

$R_i \leftarrow R_i + kR_j$ is equivalent to multiplying by the identity matrix but with $A_{ij} = k$

General Interchange Matrix

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

$R_i \leftrightarrow R_j$ is equivalent to multiplying by the identity matrix but with R_i and R_j swapped

Elementary Matrices

Df. An **elementary matrix** is a matrix obtained by applying a single row operation to the identity matrix I

$$R_i \leftarrow R_i + cR_j$$

$$E_{R_i \leftarrow R_i + cR_j}$$

Products of Elementary Matrices

Products of Elementary Matrices

Elementary matrices implement elementary row operations

Products of Elementary Matrices

Elementary matrices implement elementary row operations

matrix multiplication is transformation composition (i.e., do one then the other)

Products of Elementary Matrices

Elementary matrices implement elementary row operations

matrix multiplication is transformation composition (i.e., do one then the other)

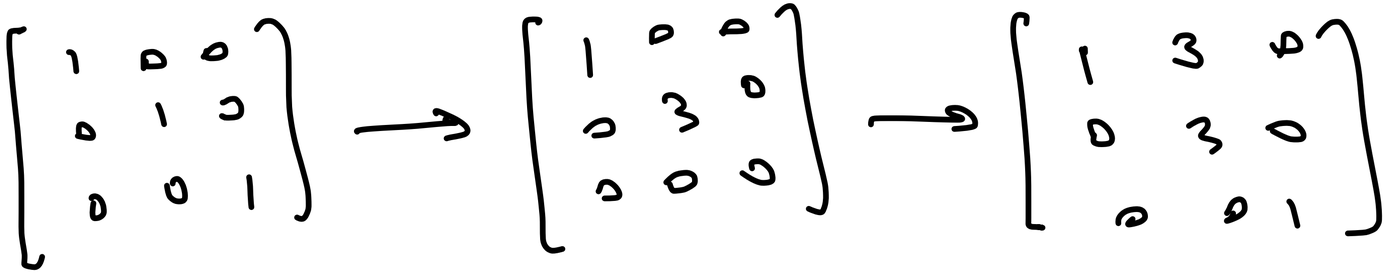
So we can implement *any* sequence of row operations as a product of elementary matrices

How to: Matrices implementing Row Operations

Q. Find the matrix implementing a sequence of row operations op_1, op_2,...

A. Apply the row operations in sequence to the identity matrix of the appropriate size

Question

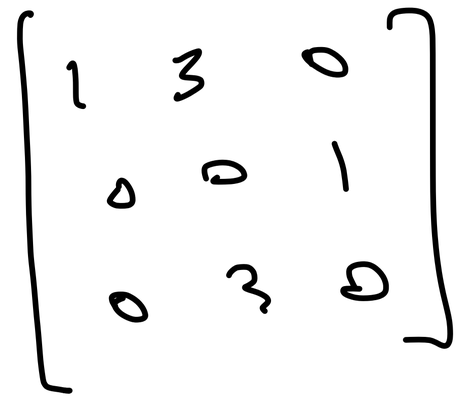


Find the matrix implementing the following sequence of elementary row operations on a $3 \times n$ *matrix*

$$R_2 \leftarrow 3R_2$$

$$R_1 \leftarrow R_1 + R_2$$

$$R_2 \leftrightarrow R_3$$



Then multiply it with the all-ones 3×3 *matrix*

Answer

$$R_2 \leftarrow 3R_2$$

$$R_1 \leftarrow R_1 + R_2$$

$$R_2 \leftrightarrow R_3$$

Another Key Observation

Another Key Observation

Elementary row operations are **invertible** linear transformations

Another Key Observation

Elementary row operations are **invertible** linear transformations

This also means the product of elementary matrices is invertible

$$(E_1E_2E_3E_4)^{-1} = E_4^{-1}E_3^{-1}E_2^{-1}E_1^{-1}$$

Note: the order reverses

Question (Conceptual)

Describe the inverse transformation for each elementary row operation

Question (Conceptual)

Describe the inverse transformation for each elementary row operation

The inverse of scaling by k is scaling by $1/k$

Question (Conceptual)

Describe the inverse transformation for each elementary row operation

The inverse of scaling by k is scaling by $1/k$

The inverse of $R_i \leftarrow R_i + kR_j$ is $R_i \leftarrow R_i - kR_j$

Question (Conceptual)

Describe the inverse transformation for each elementary row operation

The inverse of scaling by k is scaling by $1/k$

The inverse of $R_i \leftarrow R_i + kR_j$ is $R_i \leftarrow R_i - kR_j$

The inverse of swapping is swapping again

Recall: Elementary Row Operations

scaling	multiply a row by a number
interchange	switch two rows
replacement	add a scaled equation to another

Recall: Elementary Row Operations

We only need these two for the forward phase

interchange	switch two rows
replacement	add a scaled equation to another

Recall: Elementary Row Operations

We'll assume for simplicity we only need this

`replacement` add a scaled equation to another

Recall: LU Factorization

Given $A \in \mathbb{R}^{m \times n}$, we'll write A as

echelon form of A

$$A = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ * & 1 & 0 & 0 \\ * & * & 1 & 0 \\ * & * & * & 1 \end{bmatrix}}_{L} \underbrace{\begin{bmatrix} \blacksquare & * & * & * & * \\ 0 & \blacksquare & * & * & * \\ 0 & 0 & 0 & \blacksquare & * \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_{U}$$

Gaussian Elimination and Elementary Matrices

$$A \sim A_1 \sim A_2 \sim \cdots \sim \underbrace{U}_{\text{Echelon form}}$$

$$A \sim E_1 A \sim E_2 E_1 A \sim \cdots \sim E_k \cdots E_1 A$$

invertible

$$(E_k \cdots E_1)A = U$$

$$E = E_1^{-1} E_2^{-1} \cdots E_k^{-1}$$

$$EA = U \Rightarrow A = \underbrace{E^{-1}}_{L} U$$

Gaussian Elimination and Elementary Matrices

$$A \sim A_1 \sim A_2 \sim \ldots \sim A_k$$

Consider a sequence of elementary row operations from A to an echelon form

Each step can be represent as a **product with an elementary matrix**

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \ldots \sim E_k E_{k-1} \ldots E_2 E_1 A$$

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \ldots \sim E_k E_{k-1} \ldots E_2 E_1 A$$

This exactly tells us that if B is the final echelon form we get then

$$B = (E_k E_{k-1} \ldots E_2 E_1) A = EA$$

where E implements a sequence of row operations. So:

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \ldots \sim E_k E_{k-1} \ldots E_2 E_1 A$$

This exactly tells us that if B is the final echelon form we get then

Invertible

$$B = (E_k E_{k-1} \ldots E_2 E_1) A = EA$$

where E implements a sequence of row operations. So:

Gaussian Elimination and Elementary Matrices

$$A \sim E_1 A \sim E_2 E_1 A \sim \ldots \sim E_k E_{k-1} \ldots E_2 E_1 A$$

This exactly tells us that if B is the final echelon form we get then

Invertible

$$B = (E_k E_{k-1} \ldots E_2 E_1) A = EA$$

where E implements a sequence of row operations. So:

$$A = E^{-1} B = (E_1^{-1} E_2^{-1} \ldots E_{k-1}^{-1} E_k^{-1}) B$$

LU Factorization Algorithm

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
6          E ← the matrix implementing OP
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
6          E ← the matrix implementing OP
7          L ← L @ E^-1    # note the multiplication on the right
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
6          E ← the matrix implementing OP
7          L ← L @ E^-1    # note the multiplication on the right
8      RETURN (L, U)
```

LU Factorization Algorithm

```
1  FUNCTION LU_Factorization(A):
2      L ← identity matrix
3      U ← A
4      convert U to an echelon form by GE forward step # without swaps
5      FOR each row operation OP in the prev step:
6          E ← the matrix implementing OP
7          L ← L @ E^-1     # note the multiplication on the right
8      RETURN (L, U)
```

we'll see how to do this more efficiently

The forward part of Gaussian elimination is a matrix factorization

The "L" Part

$$E = E_k E_{k-1} \cdots E_2 E_1$$

This a product of elementary matrices

So $L = E^{-1} = E_1^{-1} E_2^{-1} \ldots E_{k-1}^{-1} E_k^{-1}$ **Note: the order reverses**

We won't prove this, but it's worth thinking about: **why is this lower triangular?**

And can we build this in a more efficient way?

demo

How To: LU Factorization by hand

Q. Find a LU Factorization for the matrix A (assuming no swaps)

A.

» Start with L as the identity matrix
» Find U by the forward part of GE
» For each operation $R_i \leftarrow R_i + kR_j$, set L_{ij} to $-k$

Practice Problem

$$\begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}$$

Determine an LU factorization of the above matrix using this procedure

Answer

$$\begin{bmatrix} 1 & 2 \\ 3 & 7 \end{bmatrix}$$

Analyzing LU Factorization

Analyzing Linear Algebra Algorithms

Analyzing Linear Algebra Algorithms

We will not use $O(\cdot)$ notation!

Analyzing Linear Algebra Algorithms

We will not use $O(\cdot)$ notation!

For numerics, we care about number of **FL**oating-oint **OP**erations (FLOPs):

- addition
- subtraction
- multiplication
- division
- square root

Analyzing Linear Algebra Algorithms

We will not use $O(\cdot)$ notation!

For numerics, we care about number of **FL**oating-oint **OP**erations (FLOPs):

- >> addition
- >> subtraction
- >> multiplication
- >> division
- >> square root

$2n$ vs. n is very different when $n \sim 10^{20}$

Dominant Terms

Dominant Terms

That said, we don't care about *exact* bounds

Dominant Terms

That said, we don't care about *exact* bounds

A function $f(n)$ is ***asymptotically equivalent*** to $g(n)$ if

$$\lim_{i \to \infty} \frac{f(i)}{g(i)} = 1$$

Dominant Terms

That said, we don't care about *exact* bounds

A function $f(n)$ is ***asymptotically equivalent*** to $g(n)$ if

$$\lim_{i \to \infty} \frac{f(i)}{g(i)} = 1$$

For polynomials, they are equivalent to their dominant term

Dominant Terms

the dominant term of a polynomial is the monomial with the highest degree

$$\lim_{i \to \infty} \frac{3x^3 + 100000x^2}{3x^3} = 1$$

$3x^3$ dominates the function even though the coefficient for x^2 is so large

How To: Solving systems with the LU

Q. Solve the equation $A\mathbf{x} = \mathbf{b}$ given that $A = LU$ is a LU factorization

A. First solve $L\mathbf{x} = \mathbf{b}$ to get a solution $\mathbf{c}$, then solve $U\mathbf{x} = \mathbf{c}$ to get a solution $\mathbf{d}$

How To: Solving systems with the LU

Q. Solve the equation $A\mathbf{x} = \mathbf{b}$ given that $A = LU$ is a LU factorization

A. First solve $L\mathbf{x} = \mathbf{b}$ to get a solution $\mathbf{c}$, then solve $U\mathbf{x} = \mathbf{c}$ to get a solution $\mathbf{d}$

Why is this better than just solving $A\mathbf{x} = \mathbf{b}$***?***

FLOPs for Solving General Systems

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

Matrix Inversion: $\sim 2n^3$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

Matrix Inversion: $\sim 2n^3$ FLOPS

Matrix–Vector Multiplication: $\sim 2n^2$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

Matrix Inversion: $\sim 2n^3$ FLOPS

Matrix–Vector Multiplication: $\sim 2n^2$ FLOPS

Solving by matrix inversion: $\sim 2n^3$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPs for Solving General Systems

Gaussian Elimination: $\sim \frac{2n^3}{3}$ FLOPS

GE Forward: $\sim \frac{2n^3}{3}$ FLOPS

GE Backward: $\sim 2n^2$ FLOPS

Matrix Inversion: $\sim 2n^3$ FLOPS

Matrix–Vector Multiplication: $\sim 2n^2$ FLOPS

Solving by matrix inversion: $\sim 2n^3$ FLOPS

Solving by Gaussian elimination: $\sim \frac{2n^3}{3}$ FLOPS

The following FLOP estimates are based on $n \times n$ matrices

FLOPS for solving LU systems

FLOPS for solving LU systems

LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

FLOPS for solving LU systems

LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

Solving $L\mathbf{x} = \mathbf{b}$: $\sim 2n^2$ FLOPS (by "forward" substitution)

FLOPS for solving LU systems

LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

Solving $L\mathbf{x} = \mathbf{b}$: $\sim 2n^2$ FLOPS (by "forward" substitution)

Solving $U\mathbf{x} = \mathbf{c}$: $\sim 2n^2$ FLOPS (by back substitution)

FLOPS for solving LU systems

LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

Solving $L\mathbf{x} = \mathbf{b}$: $\sim 2n^2$ FLOPS (by "forward" substitution)

Solving $U\mathbf{x} = \mathbf{c}$: $\sim 2n^2$ FLOPS (by back substitution)

Solving by LU Factorization: $\sim \frac{2n^3}{3}$ FLOPS

If you solve several matrix equations for the same matrix, LU factorization is faster than matrix inversion on the first equation, and the same (asymptotically) in later equations (and it works for rectangular matrices).

Other Considerations: Density

Other Considerations: Density

If A doesn't have to many entries (A is **sparse**), then its likely that L and U won't either.

Other Considerations: Density

If A doesn't have to many entries (A is **sparse**), then its likely that L and U won't either.

But A^{-1} may have ***many*** entries (A^{-1} is **dense**)

Other Considerations: Density

If A doesn't have to many entries (A is **sparse**), then its likely that L and U won't either.

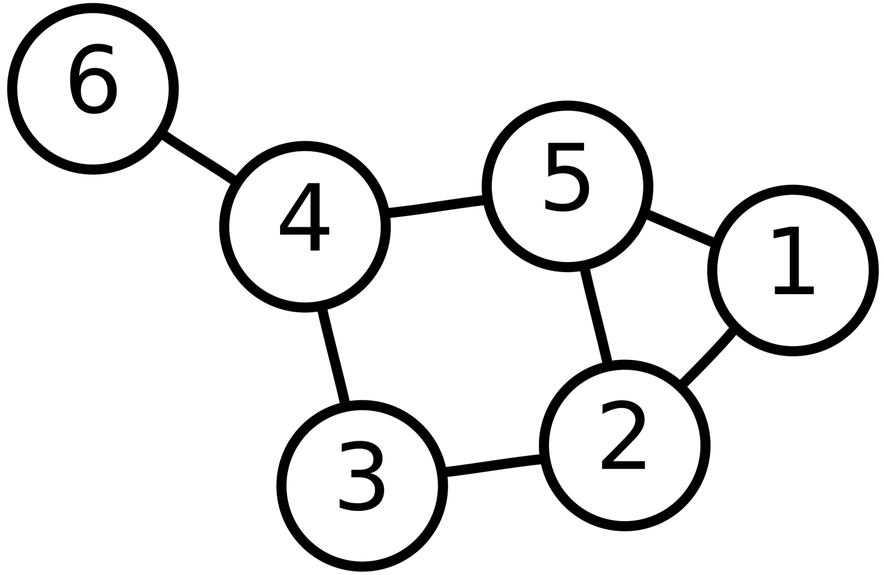
But A^{-1} may have ***many*** entries (A^{-1} is **dense**)

Sparse matrices are faster to compute with and better with respect to storage

Algebraic Graph Theory

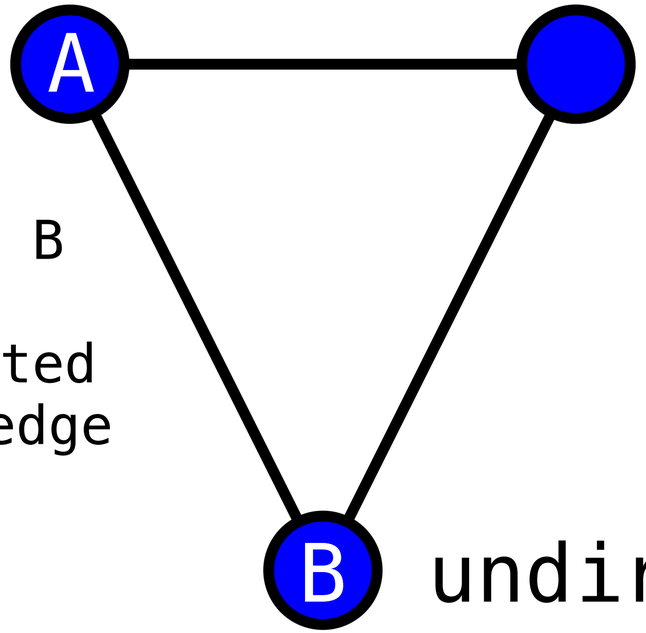
Graphs

Definition (Informal). A **graph** is a collection of nodes with edges between them.



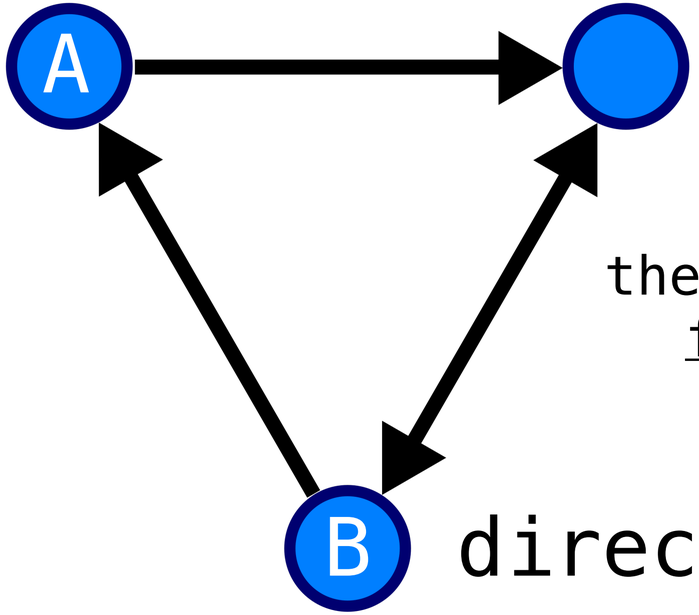
Directed vs. Undirected Graphs

A graph is **directed** if its edges have a direction.



A and B are connected by an edge

undirected

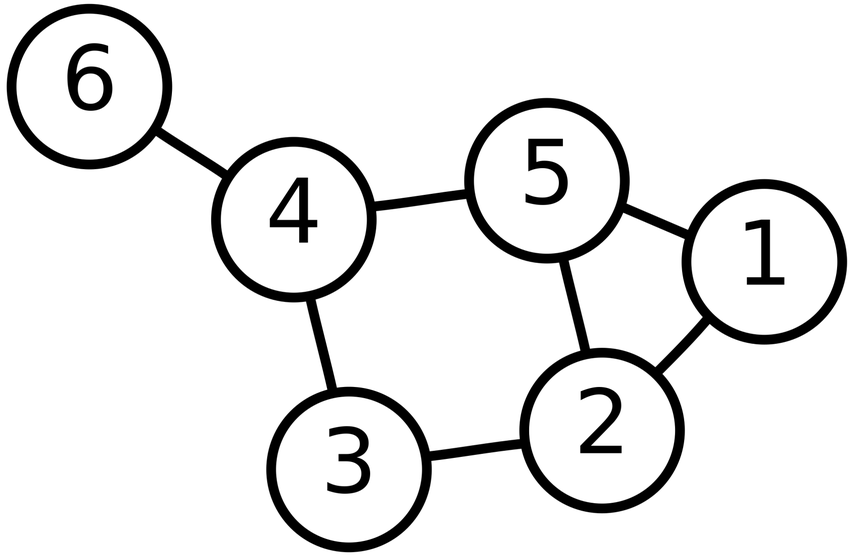


there is an edge from B to A

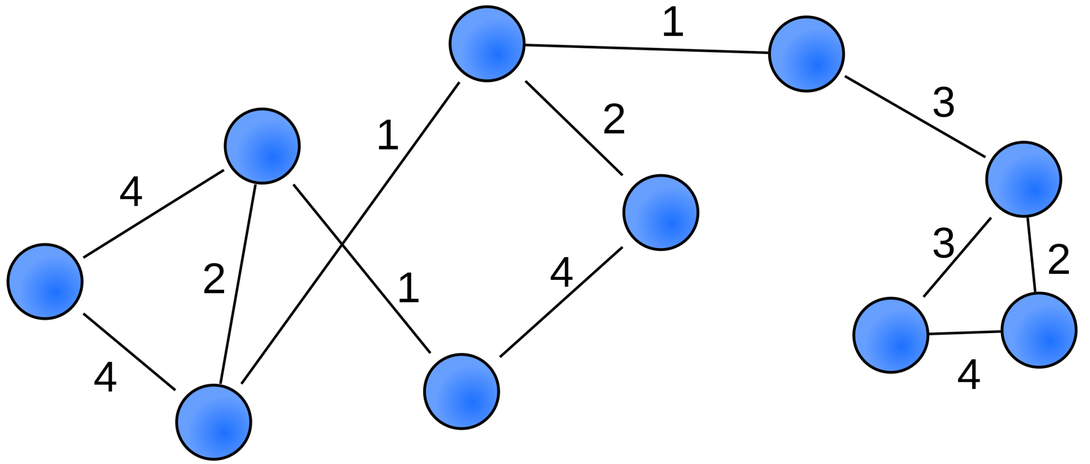
directed

Weighted vs Unweighted graphs

A graph is **weighted** if its edges have associated values.



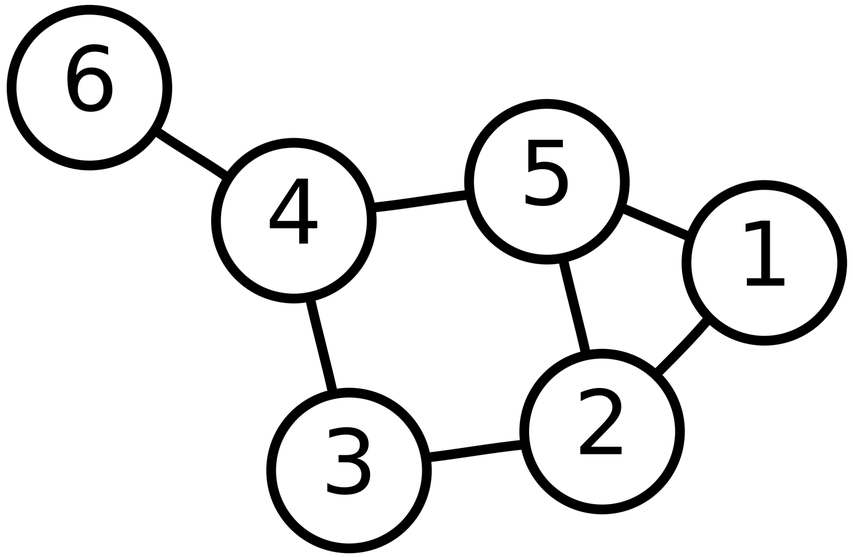
unweighted



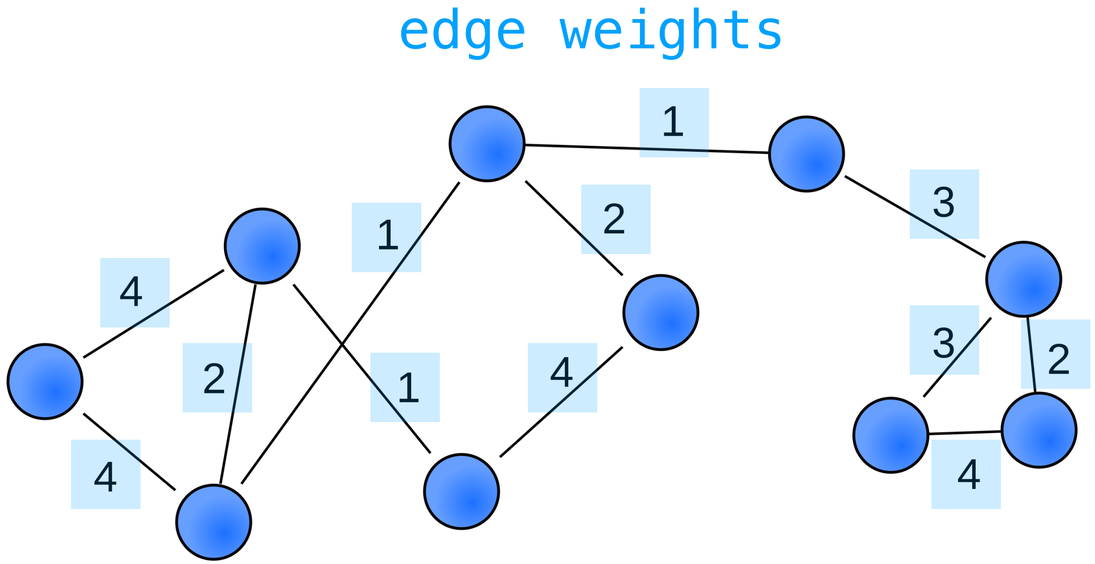
weighted

Weighted vs Unweighted graphs

A graph is **weighted** if its edges have associated values.



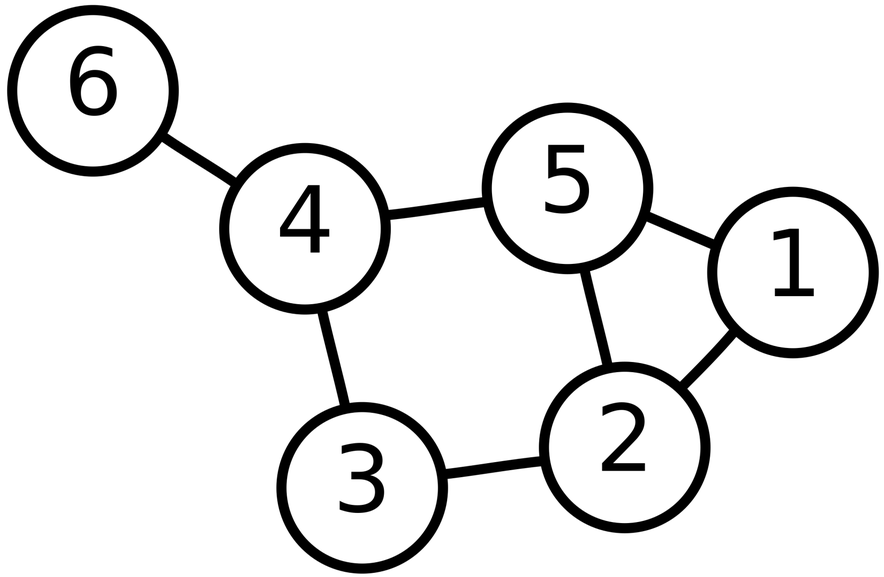
unweighted



weighted

Simple Graphs

A graph is **simple** if it is undirected, has no self loops, and no multi-edges.



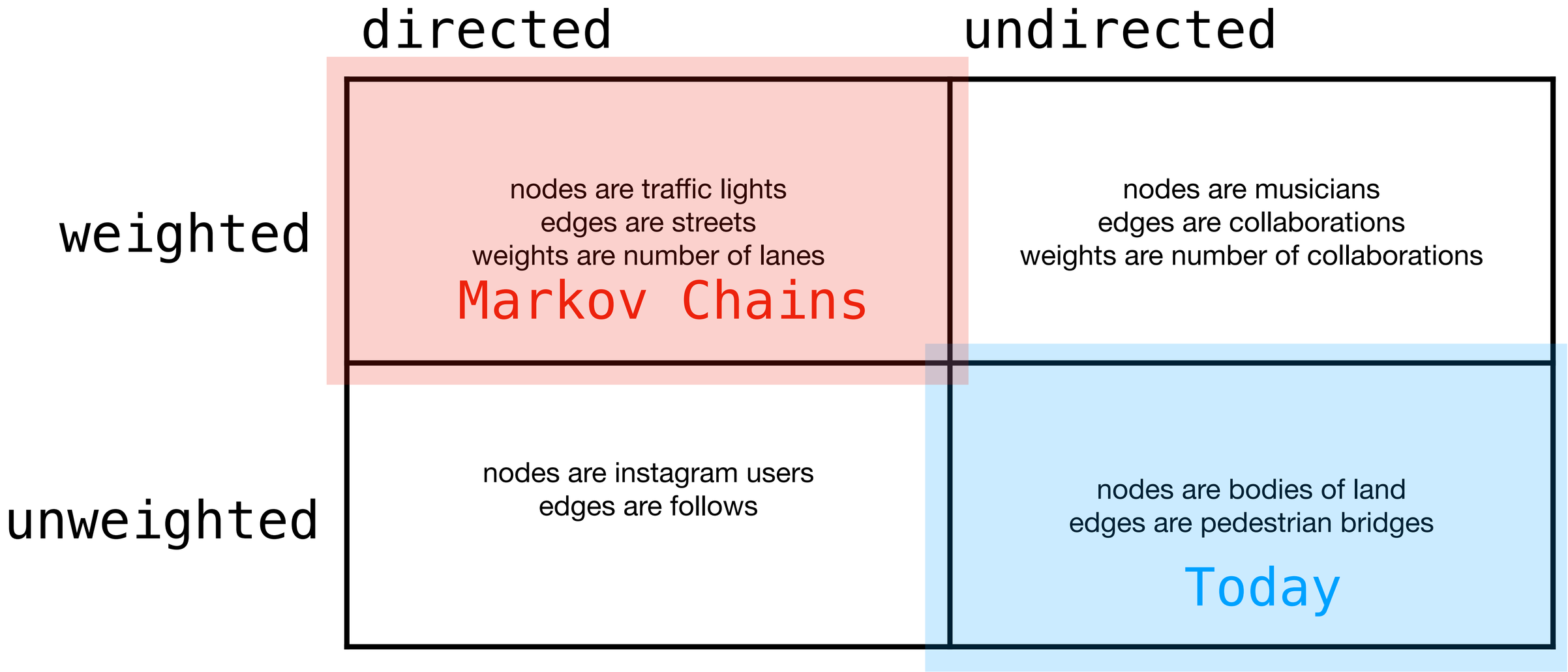
Four Kinds of Graphs

	directed	undirected
weighted	nodes are traffic lights edges are streets weights are number of lanes	nodes are musicians edges are collaborations weights are number of collaborations
unweighted	nodes are instagram users edges are follows	nodes are bodies of land edges are pedestrian bridges

Four Kinds of Graphs

	directed	undirected
weighted	nodes are traffic lights edges are streets weights are number of lanes	nodes are musicians edges are collaborations weights are number of collaborations
unweighted	nodes are instagram users edges are follows	nodes are bodies of land edges are pedestrian bridges Today

Four Kinds of Graphs



Fundamental Question

Fundamental Question

How do we represent a graph formally in a computer?

Fundamental Question

How do we represent a graph formally in a computer?

There are a couple ways, but one way is to use matrices.

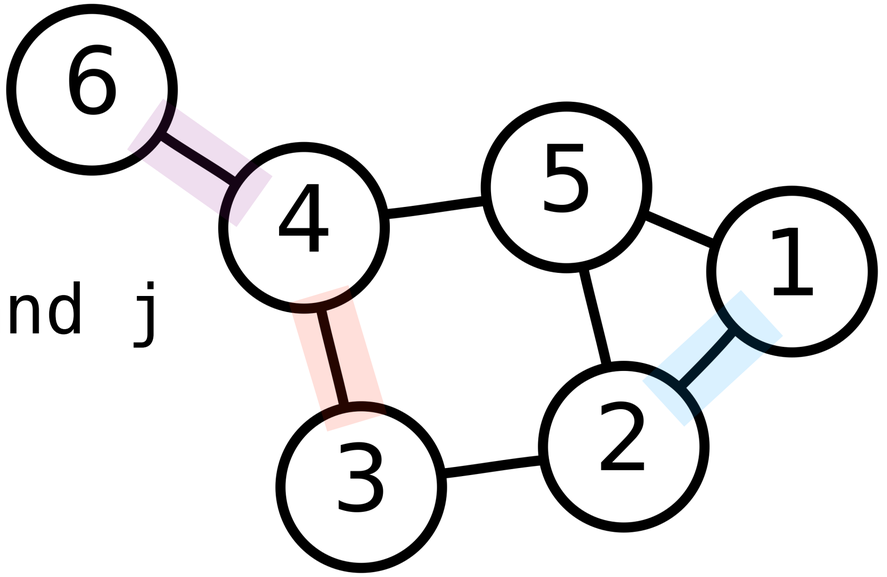
Adjacency Matrices

Let G be an simple graph with its nodes labeled by numbers 1 through n.

We can create the **adjacency matrix** A for G as follows.

$$A_{ij} = \begin{cases} 1 & \text{there is an edge between i and j} \\ 0 & \text{otherwise} \end{cases}$$

$$\begin{array}{c} \\ A_{21} \\ \\ A_{43} \\ \\ A_{64} \end{array} \begin{array}{c} \begin{array}{cccccc} A_{12} & & A_{34} & & A_{46} & \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \end{array}$$



Symmetric Matrices

Definition. A $n \times n$ matrix is **symmetric** if

$A^T = A$

Example.

$$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

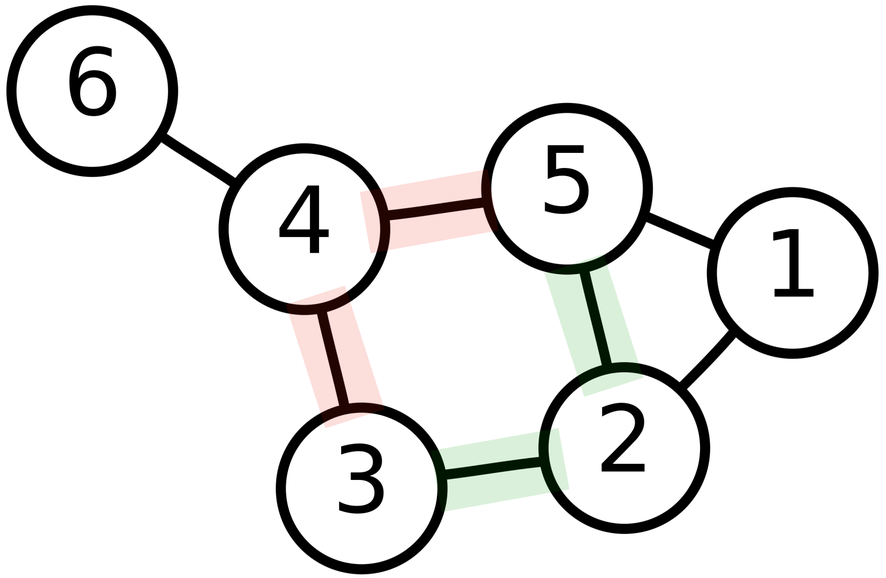
Once we have an adjacency matrix, we can do linear algebra on graphs.

Example: Squared Adjacency Matrices

Given an adjacency matrix A, can we interpret anything meaningful from A^2?

Example: Squared Adjacency Matrices

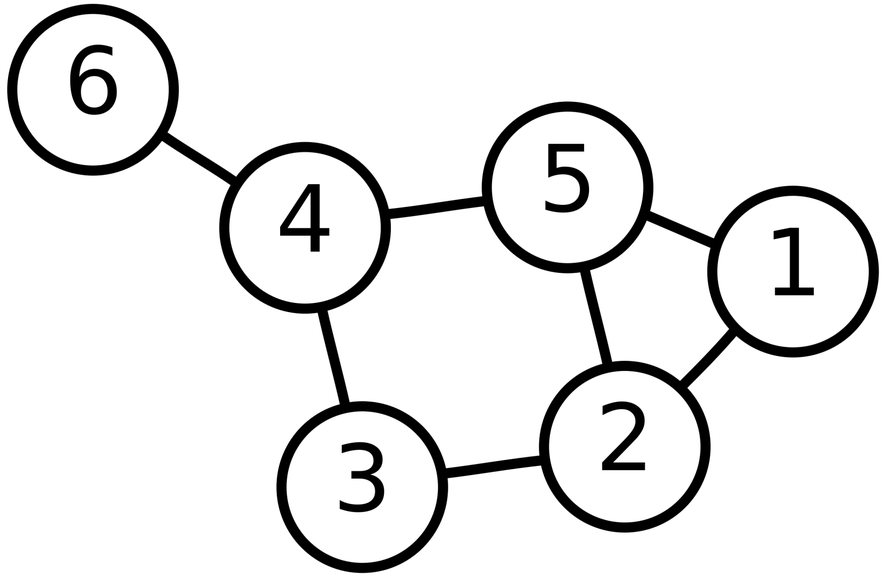
$$\begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$



$$(A^2)_{53} = 1(0) + 1(1) + 0(0) + 1(1) + 0(0) + 0(0) = 2$$

Example: Squared Adjacency Matrices

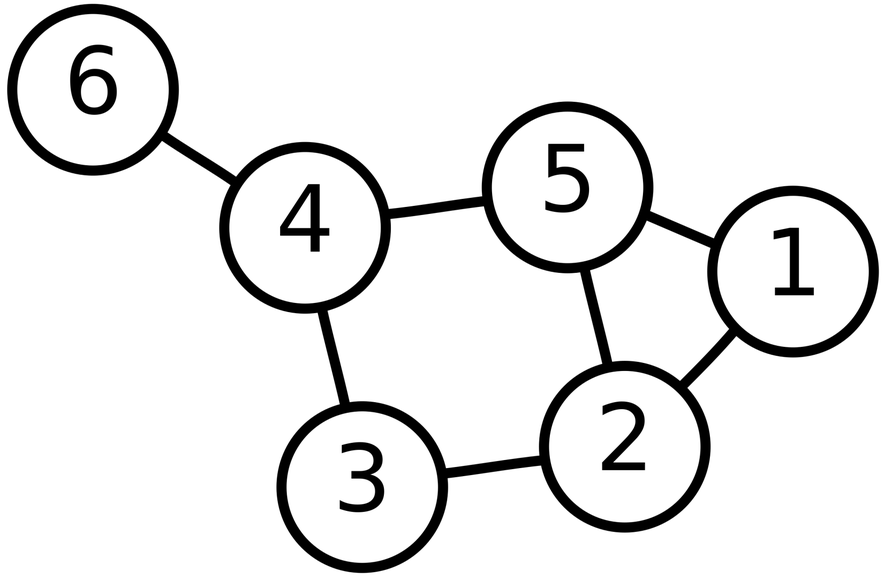
$$(A^2)_{ij} = A_{i1}A_{1j} + A_{i2}A_{2j} + \ldots + A_{in}A_{nj}$$



Example: Squared Adjacency Matrices

$$(A^2)_{ij} = A_{i1}A_{1j} + A_{i2}A_{2j} + \ldots + A_{in}A_{nj}$$

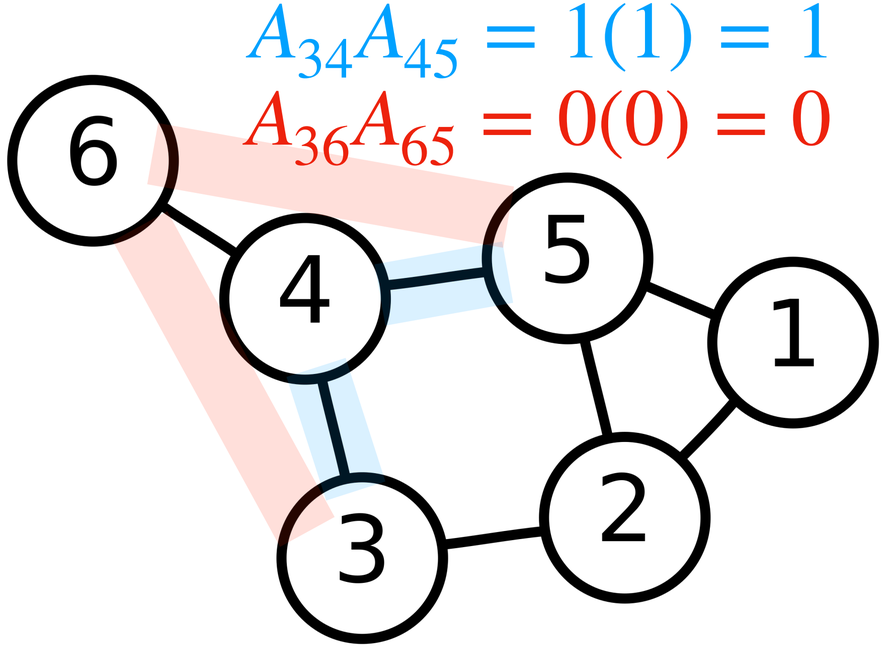
$$A_{ik}A_{kj} = \begin{cases} 1 & \texttt{there are edges i to k and k to j} \\ 0 & \texttt{otherwise} \end{cases}$$



Example: Squared Adjacency Matrices

$$(A^2)_{ij} = A_{i1}A_{1j} + A_{i2}A_{2j} + \ldots + A_{in}A_{nj}$$

$$A_{ik}A_{kj} = \begin{cases} 1 & \texttt{there are edges i to k and k to j} \\ 0 & \texttt{otherwise} \end{cases}$$

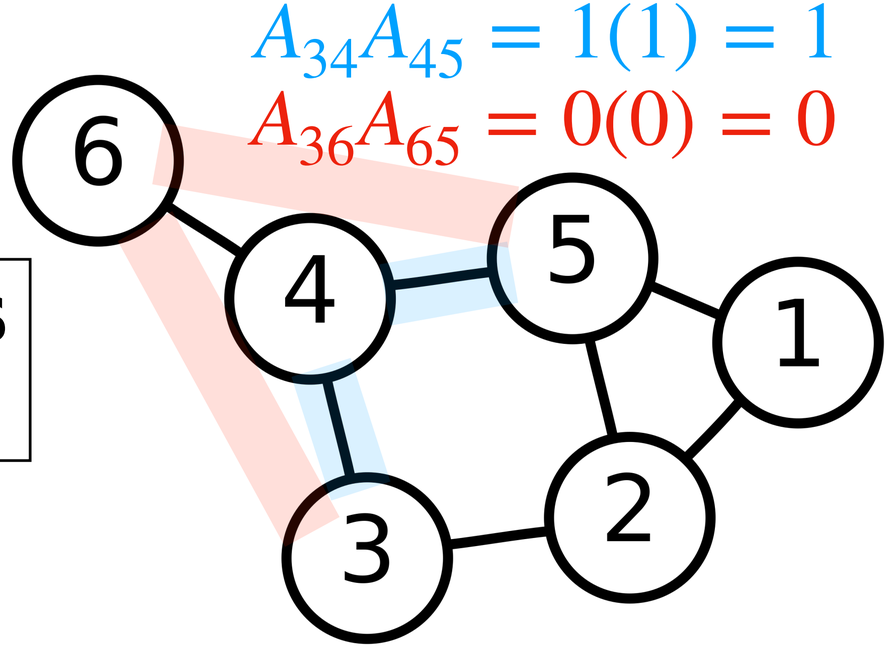


Example: Squared Adjacency Matrices

$$(A^2)_{ij} = A_{i1}A_{1j} + A_{i2}A_{2j} + \ldots + A_{in}A_{nj}$$

$$A_{ik}A_{kj} = \begin{cases} 1 & \texttt{there are edges i to k and k to j} \\ 0 & \texttt{otherwise} \end{cases}$$

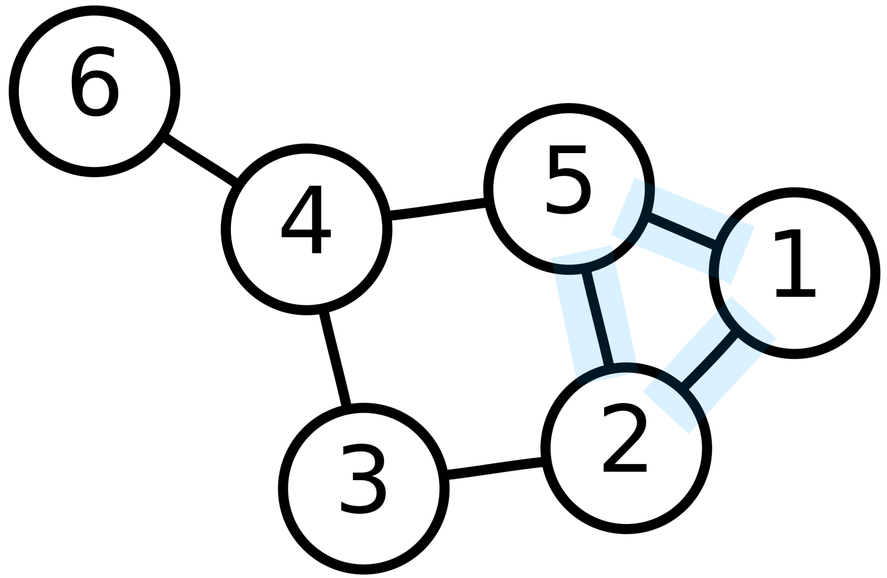
$$(A^2)_{ij} = \boxed{\texttt{number of 2-step paths from i to j}}$$



Application: Triangle Counting

A **triangle** in an undirected graph is a set of three distinct nodes with edges between every pair of nodes.

Triangles in a social network represent mutual friends and tight cohesion (among other things)



Application: Triangle Counting (Naive)

```
FUNCTION tri_count_naive(A):
  count = 0
  for i from 1 to n:
    for j from i + 1 to n:
      for k from j + 1 to n:
        if A_ij = 1 and A_jk = 1 and A_ki = 1: # an edge between each pair
          count += 1:
  RETURN count
```

Application: Triangle Counting

Theorem. For an adjacency matrix A, the number of triangle containing the edge (i,j) is

$$(A^2)_{ij} * A_{ij}$$

Verify:

Application: Triangle Counting

FUNCTION tri_count(A):

  compute A^2

  count $\leftarrow$ sum of $(A^2)_{ij} * A_{ij}$ for all distinct i and j

  RETURN count / 6 # why divided by 6?

Application: Triangle Counting

```
FUNCTION tri_count(A):
    # in NumPy '*' is entry-wise multiplication
    #     also called the HADAMARD PRODUCT
    count ← sum of the entries of A^2 * A
    RETURN count / 6
```

Application: Triangle Counting

```
FUNCTION tri_count(A):
  # in NumPy '*' is entry-wise multiplication
  #     also called the HADAMARD PRODUCT
  # and 'np.sum' sums the entry of a matrix
  RETURN np.sum((A @ A) * A) / 6
```

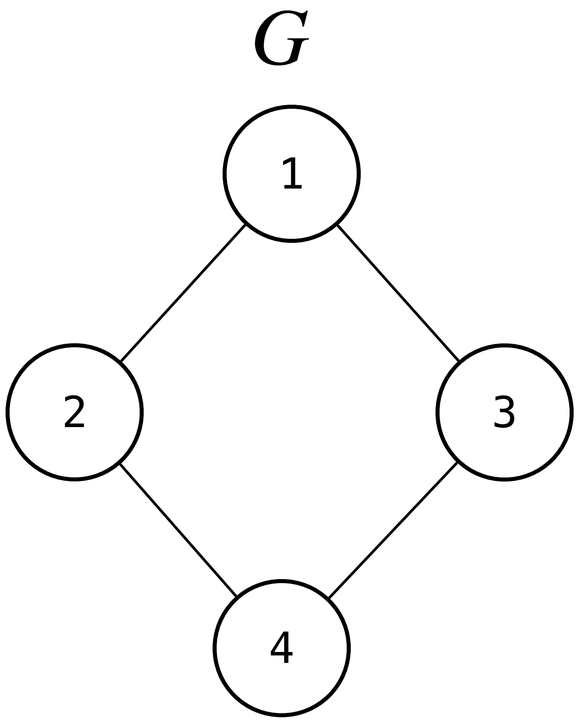
demo

Another Application: Reachability

Question: If A^2 gives us information about length 2 paths, then what about A^k?

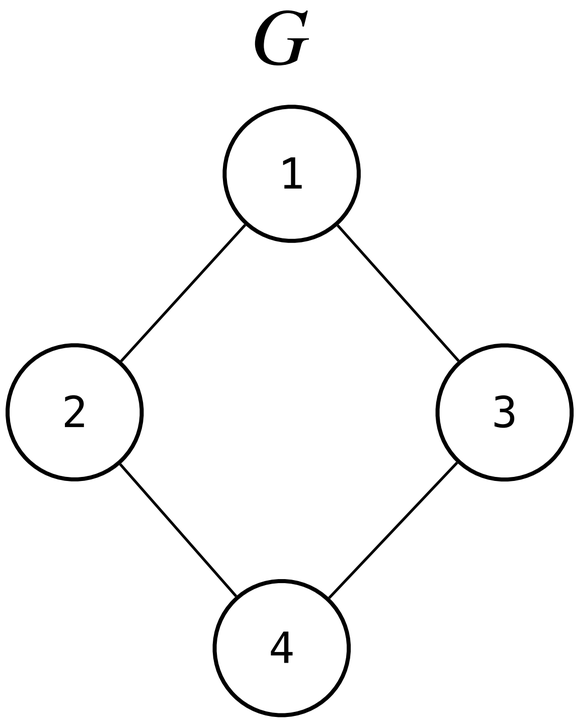
A^k gives us information about k-length paths.

Example



Example

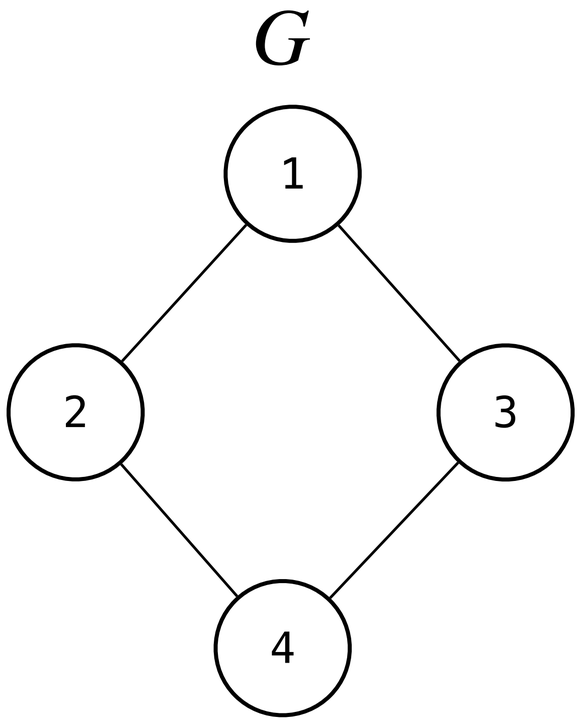
$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \text{adjacency matrix for } G$$



Example

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \text{adjacency matrix for } G$$

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}^2 = \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

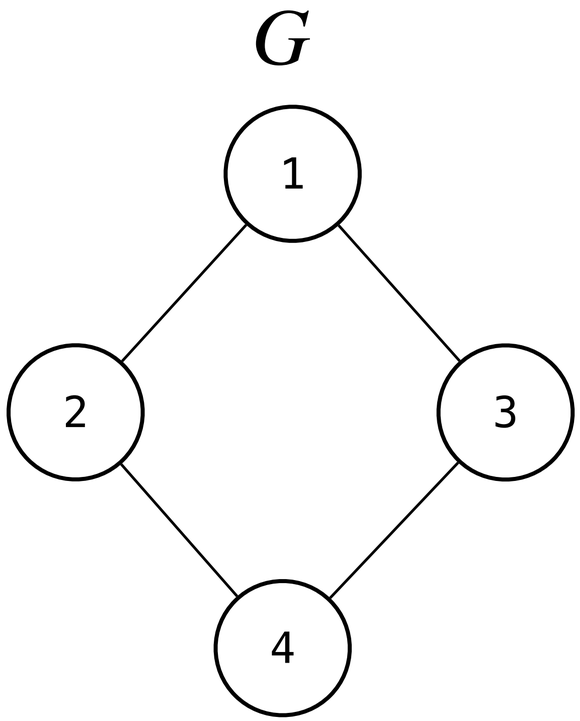


Example

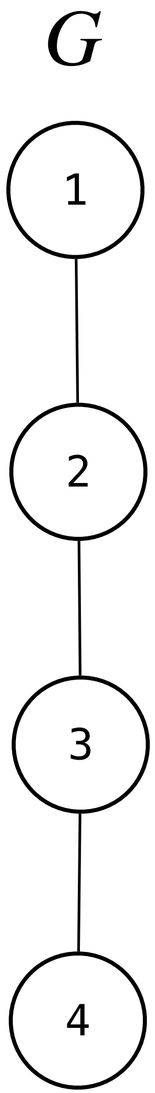
$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \text{adjacency matrix for } G$$

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}^2 = \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}^3 = \begin{bmatrix} 2 & 0 & 0 & 2 \\ 0 & 2 & 2 & 0 \\ 0 & 2 & 2 & 0 \\ 2 & 0 & 0 & 2 \end{bmatrix} \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 4 & 4 & 0 \\ 4 & 0 & 0 & 4 \\ 4 & 0 & 0 & 4 \\ 0 & 4 & 4 & 0 \end{bmatrix}$$

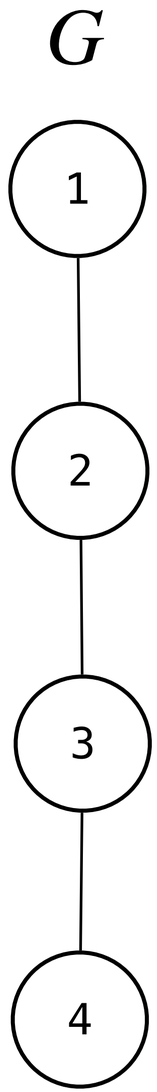


Example



Example

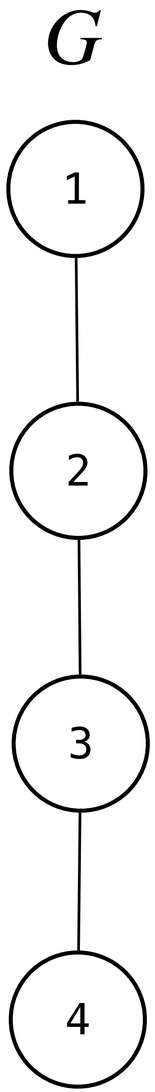
$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \text{adjacency matrix for } G$$



Example

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \text{adjacency matrix for } G$$

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}^2 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

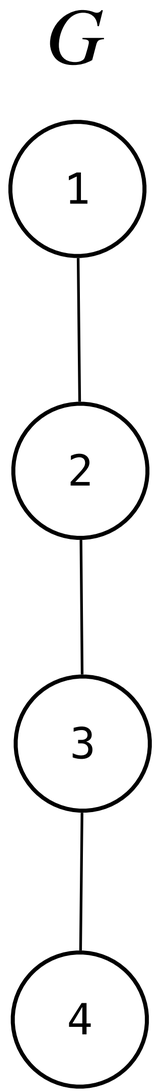


Example

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \text{adjacency matrix for } G$$

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}^2 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}^3 = \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 2 & 0 & 1 \\ 2 & 0 & 3 & 0 \\ 0 & 3 & 0 & 2 \\ 1 & 0 & 2 & 0 \end{bmatrix}$$

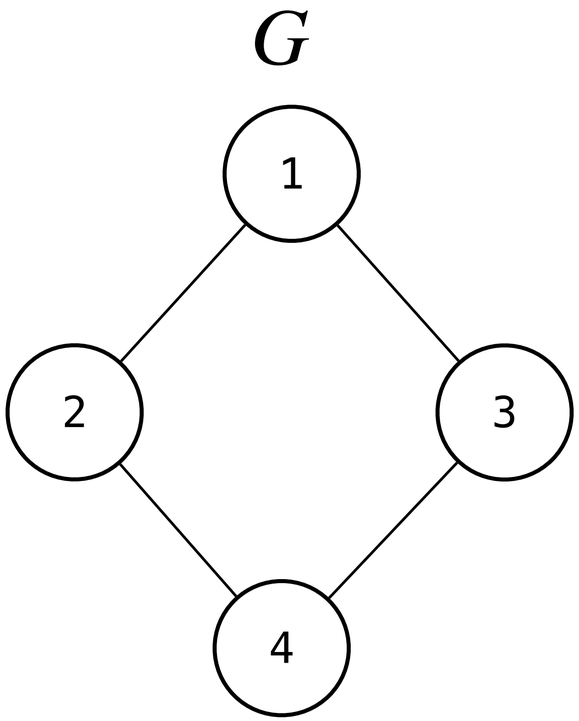


Another Application: Reachability

Theorem: Let G be a simple graph.

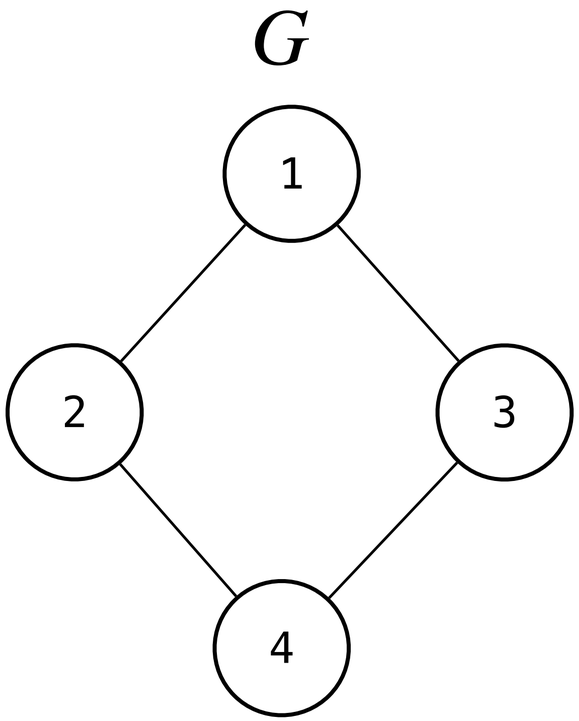
- $(A_G^k)_{ij}$ is the number of paths of length exactly k from v_i to v_j.
- $\left((A_G + I)^k\right)_{ij}$ is nonzero if and only if there is a path of length at at most k from v_i to v_j.

Example



Example

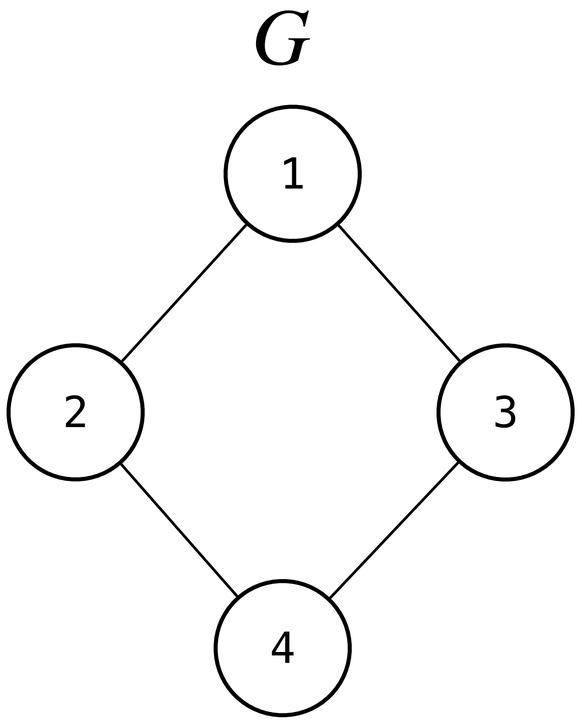
$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} = (\text{adjacency matrix for } G) + I$$



Example

$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} = (\text{adjacency matrix for } G) + I$$

$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}^2 = \begin{bmatrix} 3 & 2 & 2 & 2 \\ 2 & 3 & 2 & 2 \\ 2 & 2 & 3 & 2 \\ 2 & 2 & 2 & 3 \end{bmatrix}$$

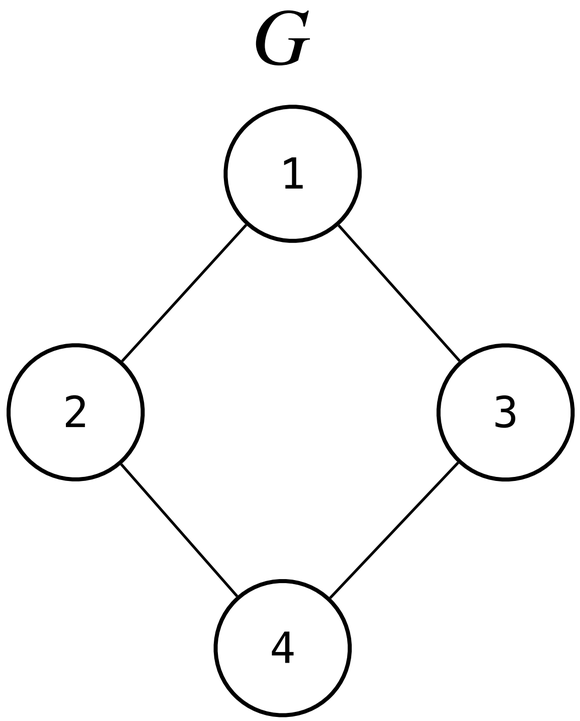


Example

$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} = (\text{adjacency matrix for } G) + I$$

$$\begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}^2 = \begin{bmatrix} 3 & 2 & 2 & 2 \\ 2 & 3 & 2 & 2 \\ 2 & 2 & 3 & 2 \\ 2 & 2 & 2 & 3 \end{bmatrix}$$

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}^3 = \begin{bmatrix} 3 & 2 & 2 & 2 \\ 2 & 3 & 2 & 2 \\ 2 & 2 & 3 & 2 \\ 2 & 2 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 6 & 6 & 6 & 7 \\ 6 & 6 & 7 & 6 \\ 6 & 7 & 6 & 6 \\ 7 & 6 & 6 & 6 \end{bmatrix}$$



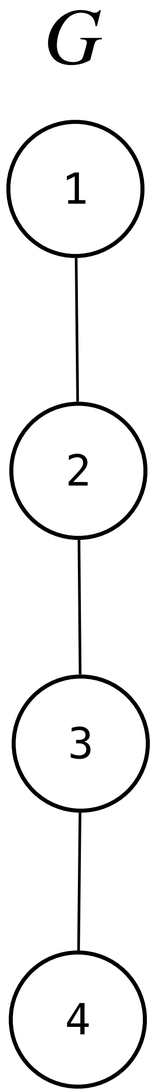
How To: Reachability

Question: Given a simple graph G determine how many nodes v_i can reach in at least k steps.

Answer: Find $(A_G + I)^k$ and count the number of nonzero elements in column i.

Question

Determine the $(A_G + I)^2$ *and* $(A_G + I)^3$ *and interpret the results.*



Summary

Matrix inverses allow us to easily solve many matrixes equations over the same A

LU Factorizations allows us to do the same, but more generally more efficiently

Adjacency matrices are linear algebraic representations of graphs